



UDEM

UNIVERSIDAD DE MONTERREY

Práctica 8

Microprocesadores

M.C. Alejandro Omar Reyes Guía

Nombre: Eugenio Romo GarcíaMat: 612348

Nombre: Ricardo Garza BenítezMat: 645972

"Damos nuestra palabra de que hemos hecho esta actividad con integridad académica"

Monterrey, N. L., a 12 de junio del 2026

Introducción

En la práctica anterior se implementó un sistema de medición sobre un único canal analógico (AN0), capaz de mostrar la lectura del ADC en tres formatos distintos: valor digital crudo, voltaje equivalente y porcentaje, alternando entre ellos mediante un botón asociado a la interrupción externa RB0. En esta práctica se extiende ese mismo concepto a un sistema de dos canales analógicos independientes (AN0 y AN1), de manera que el usuario pueda elegir tanto el canal a medir como el formato en que se despliega dicha medición, todo desde una misma pantalla LCD de dos líneas.

La actividad consiste en leer de forma continua el voltaje analógico presente en dos potenciómetros independientes, cada uno conectado a uno de los canales del ADC (AN0 y AN1). Un primer botón, conectado a RB0 y asociado a la interrupción externa INT, permite ciclar entre los tres formatos de despliegue (Voltaje, Porcentaje y ADC), tal como en la práctica anterior. Un segundo botón, conectado a RB1 y asociado a la interrupción por cambio de estado (Interrupt-On-Change, IOC), permite alternar entre el Canal 1 (AN0) y el Canal 2 (AN1). La pantalla LCD utiliza ahora sus dos líneas: en la primera se indica el canal y el modo seleccionados (por ejemplo, "Canal 1 - Volt") y en la segunda se muestra el valor numérico correspondiente.

Esta práctica integra y amplía varios conceptos vistos previamente: la configuración multicanal del módulo ADC, el manejo de una pantalla LCD a dos líneas, el uso combinado de dos mecanismos de interrupción distintos (INT e IOC) sobre el mismo puerto, y el empleo de resistencias de pull-up internas para simplificar el cableado de los botones. Adicionalmente, se corrigió la configuración del oscilador respecto a la práctica anterior, utilizando ahora el modo HS junto con `_XTAL_FREQ` de 16 MHz, en concordancia con el cristal de 16 MHz realmente instalado en la tarjeta.

Marco Teórico

Selección de canal en el módulo ADC del PIC16F887 (`ADCON0<CHS3:CHS0>`)

El módulo ADC del PIC16F887 puede convertir la señal de hasta catorce entradas analógicas distintas, pero solo puede procesar una a la vez; el canal activo se selecciona mediante los bits CHS3:CHS0 del registro `ADCON0`, ubicados en las posiciones 5 a 2 de dicho registro. Para habilitar varios canales como entradas analógicas es necesario, además, configurar los registros `ANSEL` y `ANSELH`; en esta práctica `ANSEL = 0x03` habilita simultáneamente los pines AN0 y AN1 como analógicos, dejando el resto del puerto disponible para uso digital. La función `ADC_Read` recibe como parámetro el número de canal deseado, limpia los bits CHS previos con una máscara (`ADCON0 &= 0x83`) y escribe el nuevo canal recorriéndolo dos posiciones a la izquierda (`channel << 2`). Después de cambiar de canal es necesario un breve tiempo de adquisición, ya que el capacitor de muestreo interno del ADC debe cargarse al nuevo voltaje de entrada antes de iniciar la conversión; por ello, `ADC_Read` incluye un retardo de 5 microsegundos (`__delay_us(5)`) antes de iniciar la conversión con `GO_nDONE = 1` [1].

Pantalla LCD HD44780/LM016L de dos líneas

El controlador HD44780, utilizado por la pantalla LM016L, organiza su memoria de despliegue (DDRAM) en dos bloques de direcciones correspondientes a cada línea visible del LCD. La función `LCD_Set_Cursor(fila, columna)` de la librería traduce estas coordenadas a la dirección de DDRAM correspondiente y la envía como comando al controlador antes de escribir los caracteres. En esta práctica se aprovechan ambas líneas de la pantalla: la primera línea, posicionada con `LCD_Set_Cursor(0,0)`, se utiliza para indicar de forma compacta el canal y el modo de medición activos ("Canal 1 - Volt", "Canal 2 - Porc", etc.), mientras que la segunda línea, posicionada con `LCD_Set_Cursor(1,0)`, despliega el valor numérico medido. Esta separación de información en dos renglones mejora la legibilidad respecto al esquema de un solo renglón utilizado en la práctica anterior, sin requerir ninguna modificación al cableado físico de la pantalla, que continúa operando en modo de 4 bits sobre el puerto C [2].

Interrupción externa INT (RB0) e interrupción por cambio de estado IOC (RB1)

El PIC16F887 ofrece dos mecanismos distintos para generar una interrupción a partir de un cambio en una entrada digital. El primero es la interrupción externa dedicada INT, disponible únicamente en el pin RB0, cuya bandera INTF se activa según el flanco configurado en el bit INTEDG del registro `OPTION_REG`. El segundo es la interrupción por cambio de estado (Interrupt-On-Change, IOC), que permite habilitar de manera individual cualquier pin de PORTB mediante el registro IOCB, generando la bandera RBIF cuando cualquiera de los pines habilitados cambia de nivel, sin importar la dirección del cambio. En esta práctica, RB0 utiliza la interrupción INT (`INTEDG = 0`, flanco de bajada) para el cambio de modo de despliegue, mientras que RB1 utiliza la interrupción IOC (`IOCB = 0b00000010`) para el cambio de canal. Ambas fuentes comparten la misma rutina de servicio ISR, la cual se limita a limpiar la bandera correspondiente y activar una variable booleana (`mode_event` o `chan_event`); el procesamiento real, incluyendo el retardo de antirrebote, se realiza posteriormente dentro del ciclo principal, lo que mantiene la rutina de interrupción breve [1].

Resistencias de pull-up internas en PORTB (registro WPUB y bit RBPU)

Para que un botón conectado entre un pin y tierra pueda ser leído de forma confiable, dicho pin debe mantenerse en un nivel alto definido mientras el botón no esté presionado; esto se logra tradicionalmente con una resistencia de pull-up externa. El PIC16F887 incluye resistencias de pull-up internas individuales para cada pin de PORTB, las cuales se habilitan a través del registro WPUB y se activan globalmente al limpiar el bit RBPU del registro `OPTION_REG`. En esta práctica, `TRISB` se configura para que RB0 y RB1 sean entradas, WPUB habilita el pull-up en ambos pines y `OPTION_REG &= 0b01111111` limpia el bit RBPU, activando dichas resistencias internas. De esta manera, los botones de cambio de modo y de canal pueden conectarse directamente entre el pin correspondiente y tierra, sin necesidad de resistencias externas adicionales, simplificando el cableado del circuito [1] [3].

Configuración del oscilador HS y el cristal externo de 16 MHz

A diferencia de la práctica anterior, en esta ocasión la configuración del oscilador se ajustó para corresponder con el hardware disponible: se seleccionó `FOSC = HS`, el modo de oscilador recomendado por el fabricante para cristales de alta frecuencia (de 4 a 20

MHz aproximadamente), y se definió `_XTAL_FREQ = 16000000`, en concordancia con el cristal de 16 MHz instalado en la tarjeta de desarrollo. Esta correspondencia es importante por dos razones: primero, el modo HS garantiza que el circuito interno del oscilador esté diseñado para sostener la oscilación a esa frecuencia con la ganancia adecuada; segundo, al coincidir `_XTAL_FREQ` con la frecuencia real, las funciones `__delay_ms` y `__delay_us` generadas por el compilador XC8 corresponden a tiempos reales y no a una fracción o múltiplo de ellos. Esto es especialmente relevante para el retardo de antirrebote de 50 ms utilizado en ambos botones, ya que un descuido en estos valores haría que dicho retardo fuera, en la práctica, distinto al valor nominal programado [1] [3].

Simulación

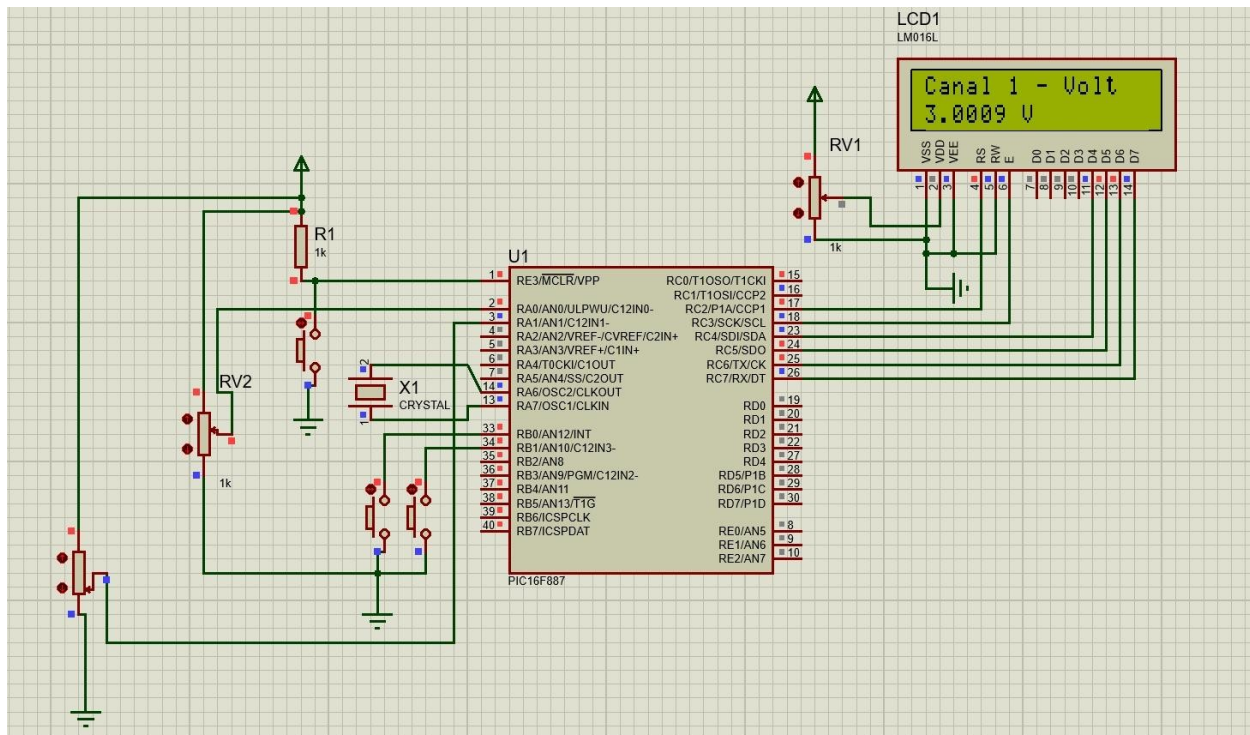
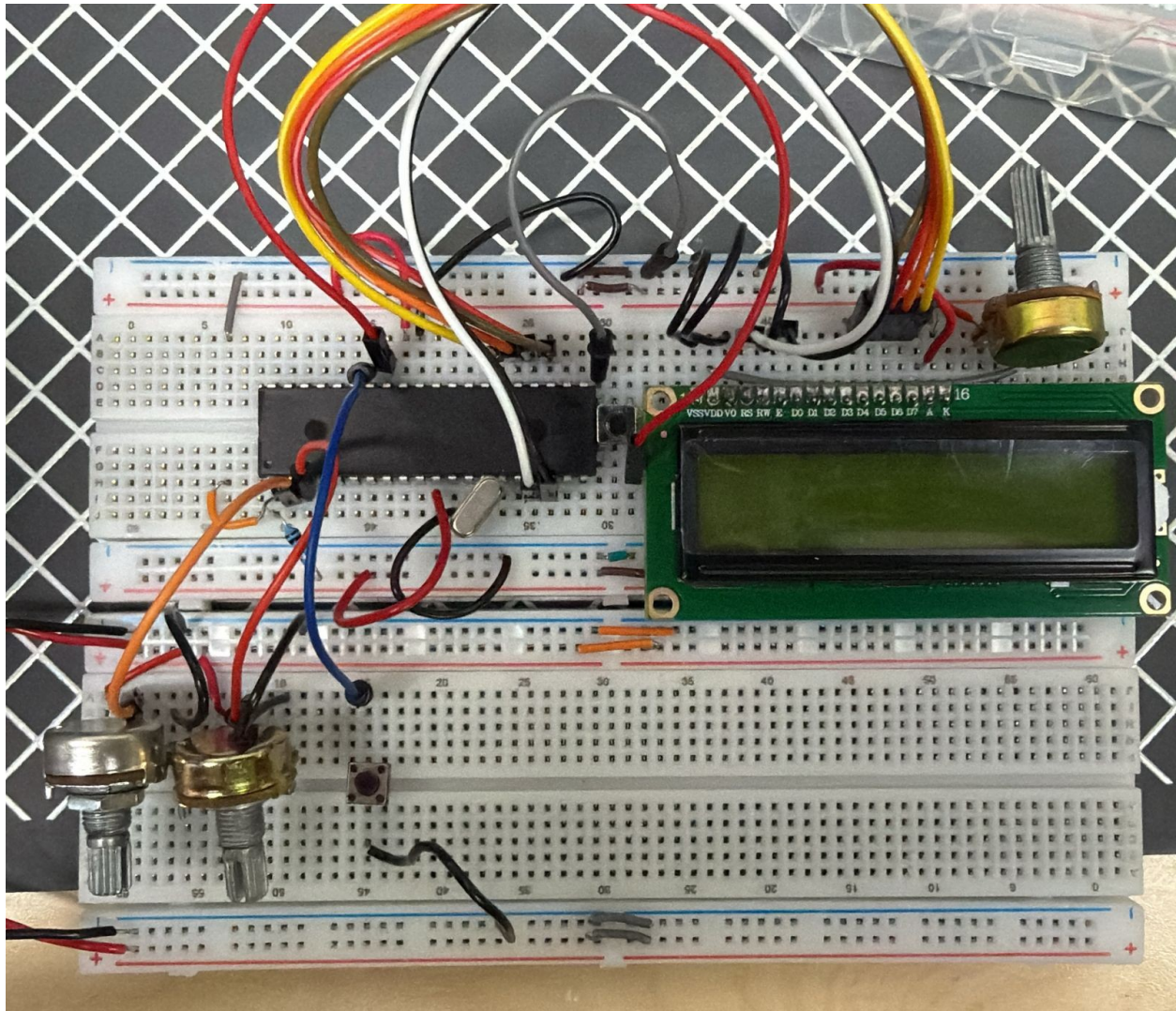


Foto del circuito en Físico



Resultados

Al energizar el circuito, la pantalla LCD mostró en su primera línea la leyenda "Canal 1 - Volt" y en la segunda línea el voltaje correspondiente al potenciómetro conectado a AN0, obteniéndose lecturas como "3.0009 V" para una posición intermedia del potenciómetro, valor coherente con la fórmula $(adc_result \times 50000) / 1023$ aplicada sobre el resultado de 10 bits del ADC. Al girar dicho potenciómetro se observó que el voltaje desplegado variaba de forma continua entre aproximadamente 0.0000 V y 4.8900 V, manteniéndose en todo momento dentro del rango esperado para una referencia de 5 V.

Al presionar el botón conectado a RB1, la pantalla cambió de "Canal 1" a "Canal 2", y el valor mostrado pasó a corresponder al segundo potenciómetro conectado a AN1, de forma totalmente independiente del primero: variar el potenciómetro del Canal 1 no alteraba la lectura mostrada del Canal 2, y viceversa, confirmando que la selección de

canal mediante los bits CHS de ADCON0 funcionaba correctamente. De manera independiente al canal seleccionado, el botón conectado a RB0 permitió ciclar entre los tres modos de despliegue ("Volt", "Porc" y "ADC"), mostrando para un mismo canal el voltaje, el porcentaje (0 % a 100 %) y la cuenta cruda (0 a 1023) de forma consistente entre sí.

Durante las primeras pruebas en físico, el circuito presentó un corto y no respondía; tras revisar la conexión se identificó que el microcontrolador estaba colocado una posición de pines corrida respecto a la posición correcta en la protoboard, lo que ocasionaba que varias señales quedaran conectadas a pines distintos a los previstos en el código. Una vez recolocado el PIC16F887 en su posición correcta, el circuito funcionó de manera correcta y estable, con un comportamiento en físico idéntico al observado en la simulación de Proteus para ambos canales y los tres modos de despliegue.

Conclusiones Individuales

Ricardo Garza

El problema más importante que enfrentamos en esta práctica fue, otra vez, de cableado y no de código: al armar el circuito en la protoboard, el PIC16F887 quedó colocado una fila de pines corrida respecto a donde debía ir, lo que provocó un corto entre líneas que en teoría no deberían estar conectadas entre sí. Lo interesante es que, al revisar el código primero, todo parecía correcto: la selección de canal con ADCON0 y los pull-ups de RB0 y RB1 estaban bien configurados, pero el circuito simplemente no respondía. Esto me reforzó la idea de que, antes de sospechar del firmware, conviene revisar la correspondencia física entre cada pin del esquemático y su posición real en la protoboard, sobre todo en circuitos con tantas conexiones como este, donde un microcontrolador corrido una sola posición puede generar cortos que no son evidentes a simple vista. Una vez corregido esto, pude probar cada canal por separado girando los potenciómetros y confirmar que el cambio de canal con RB1 y el cambio de modo con RB0 funcionaban de forma independiente, lo cual fue satisfactorio después de la práctica anterior, donde tuvimos dudas sobre la configuración del oscilador: en esta ocasión usamos FOSC = HS con _XTAL_FREQ de 16 MHz desde el principio, congruente con el cristal real de la tarjeta, y los tiempos de antirrebote de los botones se sintieron consistentes y sin pulsaciones dobles.

Eugenio Romo

Esta práctica me ayudó a entender cómo un mismo módulo ADC puede atender varias señales analógicas sin necesidad de múltiples conversores: basta con cambiar los bits CHS de ADCON0 antes de iniciar cada conversión para que el resultado corresponda al canal deseado, siempre dejando un pequeño tiempo de adquisición para que el capacitor de muestreo se ajuste al nuevo voltaje. También me quedó más claro la diferencia entre los dos tipos de interrupción usados: la interrupción externa INT en RB0, que solo puede usarse en ese pin y responde a un flanco específico, contra la interrupción por cambio de estado IOC en RB1, que puede habilitarse en cualquier pin de PORTB y responde a cualquier cambio de nivel. Ver que ambas interrupciones comparten una sola rutina de servicio, y que esta solo activa banderas que después se atienden en el ciclo principal,

me parece un patrón de diseño ordenado que probablemente volvamos a usar cuando tengamos que manejar más de dos fuentes de eventos al mismo tiempo.

Referencias

- [1] Microchip Technology. (2009). PIC16F882/883/884/886/887 Data Sheet. <https://ww1.microchip.com/downloads/en/DeviceDoc/41291D.pdf>
- [2] Hitachi. (1998). HD44780U (LCD-II) Dot Matrix Liquid Crystal Display Controller/Driver. <https://www.sparkfun.com/datasheets/LCD/HD44780.pdf>
- [3] Merino Pérez, L. A. (2012). Microcontroladores PIC: diseño práctico de aplicaciones (3.^a ed.). McGraw-Hill.